Pre-processing_FeatureEngineering_Modeling

December 1, 2025

[ ]:
```
!pip install emoji
import pandas as pd

from google.colab import drive
drive.mount('/content/drive')

# read sample data from csv
file_path = '/content/drive/My Drive/notebooks/Yelp JSON/yelp_dataset (Unzipped␣
 ↪Files)/yelp_review_data_binned.csv'
df_review_data_binned = pd.read_csv(file_path)

# Null value checking
print(df_review_data_binned.isnull().sum())
df_review_data_binned.info()
```

```
Requirement already satisfied: emoji in /usr/local/lib/python3.12/dist-packages
(2.15.0)
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
review_id         0
user_id           0
business_id       0
stars             0
useful            0
funny             0
cool              0
text              0
date              0
stars_business    0
stars_binned      0
dtype: int64
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144209 entries, 0 to 144208
Data columns (total 11 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
 0   review_id       144209 non-null  object
 1   user_id         144209 non-null  object
```

```
 2   business_id     144209 non-null  object
 3   stars           144209 non-null  int64
 4   useful          144209 non-null  int64
 5   funny           144209 non-null  int64
 6   cool            144209 non-null  int64
 7   text            144209 non-null  object
 8   date            144209 non-null  object
 9   stars_business  144209 non-null  float64
 10  stars_binned    144209 non-null  int64
dtypes: float64(1), int64(5), object(5)
memory usage: 12.1+ MB
```

[ ]:
```python
# Pre-processing
import emoji
import re
import nltk
import unicodedata
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

df_review_data_binned = df_review_data_binned.dropna(subset=['stars_binned'])
df_review_data_binned = df_review_data_binned.dropna(subset=['text'])
print(df_review_data_binned.shape)


# Identify if there are any emoji characters

def clean_text(text):
  if text is None:
    return ""
  text = emoji.replace_emoji(text, replace='')
  # convert to lower case
  text = text.lower()
  # remove strings that start with http
  text = re.sub(r'http\S+', '', text)
  # remove HTML tags
  text = re.sub(r'<.*?>', '', text)
  # remove special characters
  text = re.sub(r'www\S+', '', text)
  # remove punctuations
  text = re.sub(r'[^\w\s]', '', text)
  # remove numbers
  text = re.sub(r'\d+', '', text)
  text = unicodedata.normalize('NFKD', text)
  text = text.encode('ascii', 'ignore').decode('utf-8')
  #remove any left non-ascii characters
  text = re.sub(r'[^a-zA-Z\s]', '', text)
```

```
  # remove extra spaces
  text = re.sub(r'\s+', ' ', text).strip()

  return text

df_review_data_binned["text"] = df_review_data_binned["text"].apply(clean_text)
# download stopwords
nltk.download('stopwords')
# download tokenizer data
nltk.download('punkt_tab')
# remove stopwords
stop_words = set(stopwords.words('english'))
# remove stopwords from text column
df_review_data_binned['text'] = df_review_data_binned['text'].apply(lambda x: '␣
 ↪'.join([word for word in x.split() if word not in (stop_words)]))
# perform lemitazation
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
# download lexical database
nltk.download('wordnet')
# multilingual lexical database
nltk.download('omw-1.4')
# perform lemmatization
df_review_data_binned['text'] = df_review_data_binned['text'].apply(lambda x: '␣
 ↪'.join([lemmatizer.lemmatize(word) for word in word_tokenize(x)]))
```

```
(144209, 11)
```

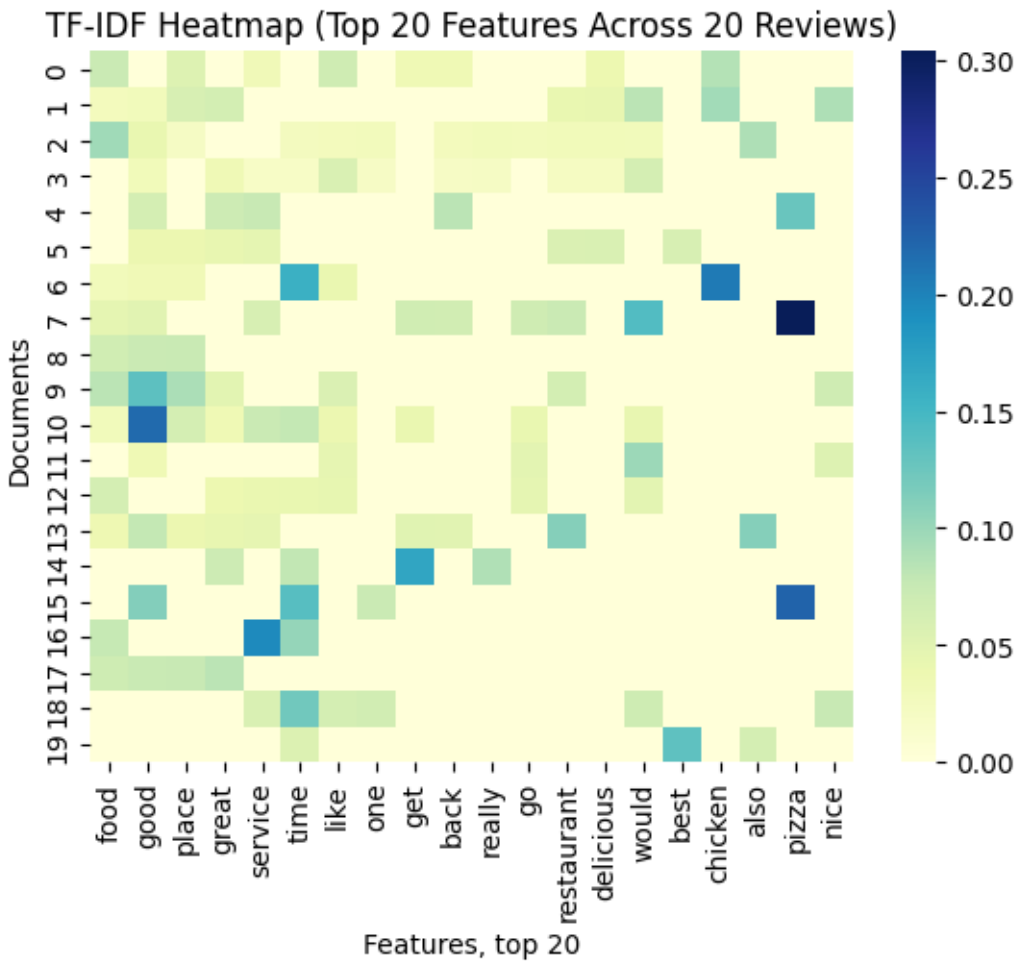
```
[nltk_data] Downloading package stopwords to /root/nltk_data…
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data…
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data…
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data…
[nltk_data]   Package omw-1.4 is already up-to-date!
```

[ ]:
```
from sklearn.model_selection import train_test_split
# define X and y
X = df_review_data_binned[["text","useful"]]
y = df_review_data_binned['stars_binned']

# Split all data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Take only text
```

```
X_train_text = X_train['text']
X_test_text = X_test['text']
```

[ ]:
```
# TfidfVectorizer (from sklearn.feature_extraction.text) converts raw text into␣
 ↪numerical feature vectors using the TF-IDF weighting scheme.
from sklearn.feature_extraction.text import TfidfVectorizer
vectorizer = TfidfVectorizer(
    min_df=5, # ignore words that appear in < 5 documents,
    max_df=0.8, # ignore words that appear in > 80% of the documents,
    max_features=10000,
    ngram_range=(1, 2))
X_train_tfidf = vectorizer.fit_transform(X_train_text.tolist())
X_test_tfidf = vectorizer.transform(X_test_text.tolist())
```

[ ]:
```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
# Plot heatmap of top 20 features
#Sum TF-IDF scores for each feature across all documents
feature_sums = np.array(X_train_tfidf.sum(axis=0)).flatten()
feature_names = np.array(vectorizer.get_feature_names_out())

# Create a pandas Series and sort
top_features = pd.Series(feature_sums, index=feature_names).
 ↪sort_values(ascending=False).head(20)

# Select top features to plot
top_feature_indices = [vectorizer.vocabulary_[f] for f in top_features.index]
X_top = X_train_tfidf[:, top_feature_indices].toarray()[:20]  # first 20 docs

# Plot heatmap
sns.heatmap(X_top, cmap="YlGnBu", xticklabels=top_features.index)
plt.title("TF-IDF Heatmap (Top 20 Features Across 20 Reviews)")
plt.xlabel("Features, top 20")
plt.ylabel("Documents")
plt.show()
```

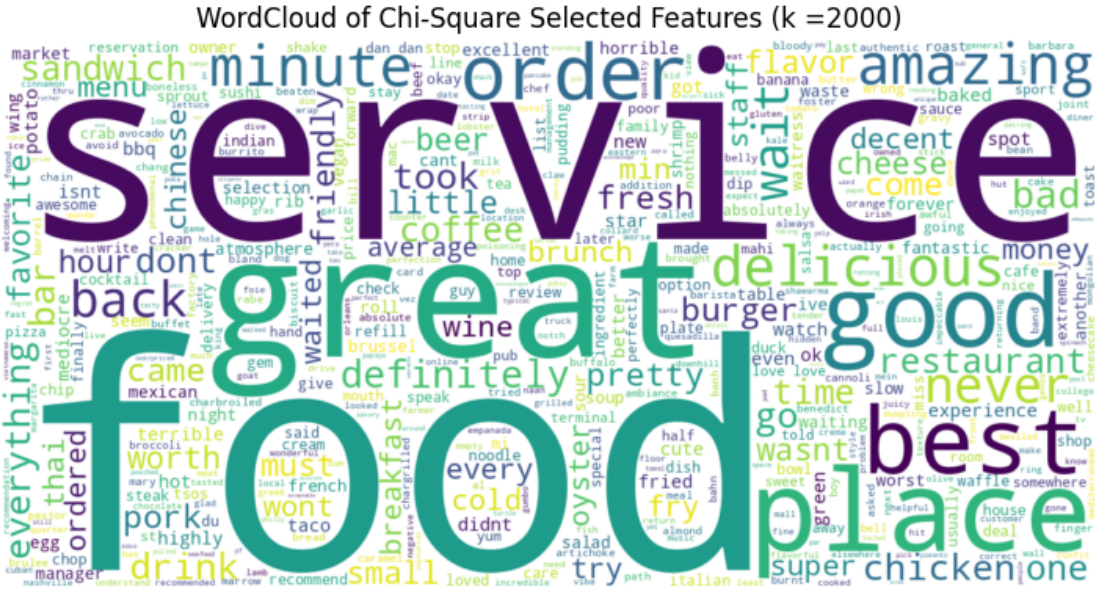


This section of the code applies **Chi2** feature selection with various values of **k**, to retain the most informative terms and visualizes selected features with **word cloud**

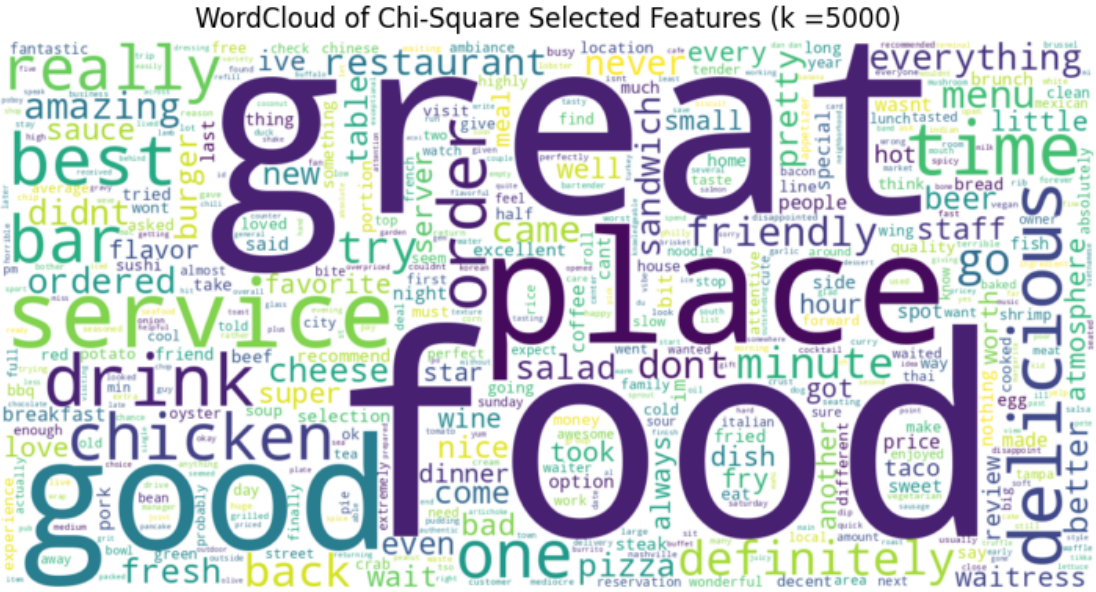
[ ]:
```
from sklearn.model_selection import train_test_split
from sklearn.feature_selection import SelectKBest, chi2
from wordcloud import WordCloud

k_values = [2000, 5000]
results = []
selected_features = {}
X_train_chi2_selection = {}
X_test_chi2_selection = {}

for k in k_values:
  # Mutual Information feature selection based, differnt numberof features␣
 ↪selected for comparison
  selector = SelectKBest(score_func=chi2, k=k)
```

```
    X_train_chi2_selection[k] = selector.fit_transform(X_train_tfidf, y_train)
    X_test_chi2_selection[k]  = selector.transform(X_test_tfidf)

    # all feature names after TF-IDF
    all_features_after_tfidf = vectorizer.get_feature_names_out()

    # Align feature names with selected indices
    selected_features[k] = all_features_after_tfidf[selector.get_support()]

    print(f"Selected features (k): {X_train_chi2_selection[k].shape[1]}")
    print(f"Shapes -> Train: {X_train_chi2_selection[k].shape}, Test:␣
    ↪{X_test_chi2_selection[k].shape}")

    selected_text = " ".join(selected_features[k])

    wordcloud = WordCloud(
        width=1200,
        height=600,
        background_color="white",
        colormap="viridis",
        max_words=500,
        random_state=42
    ).generate(selected_text)

    # Plot
    plt.figure(figsize=(9, 6))
    plt.imshow(wordcloud, interpolation="bilinear")
    plt.axis("off")
    plt.title(f"WordCloud of Chi-Square Selected Features (k ={k})", fontsize=12)
    plt.show()
    print('\n')
```

```
Selected features (k): 2000
Shapes -> Train: (115367, 2000), Test: (28842, 2000)
```




```
Selected features (k): 5000
Shapes -> Train: (115367, 5000), Test: (28842, 5000)
```



[ ]:
```
def _evaluate_feature_subset(
        model,
        X_train,
        y_train,
        X_valid,
        y_valid,
        feature_indices,
        scoring,
        score_average
):
    """Fit model on selected features and return validation score."""
    X_train_subset = X_train[:, feature_indices]
    X_valid_subset = X_valid[:, feature_indices]
    model.fit(X_train_subset, y_train)
    y_pred = model.predict(X_valid_subset)
    return scoring(y_valid, y_pred, average=score_average)
```

[ ]:
```
from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score, classification_report
from sklearn.model_selection import train_test_split
import time

def batched_forward_selection(
    X,
    y,
    n_features_to_select=300,
    batch_size=20,
    val_size=0.1,
    random_state=42,
    scoring=f1_score,
    score_average="macro",
):

    """Perform batched forward feature selection using Logistic Regression.

    Parameters
    ----------
    X : train feature matrix (n_samples, n_features)

    y : target(n_samples)

    n_features_to_select : int, default=300
    Number of features to select.

    batch_size : int, default=20
    Number of features to add per iteration.
```

```
    val_size : float, default=0.2
    breakdown of data in the validation split.

    random_state : int, default=42
    random statefor reproducibility.

    scoring : callable, default=f1_score
    scoring function to use.

    score_average : str, default="macro"
    average parameter for scoring function.

    Returns
    -------
    selected : list of int
    Indices of the selected features.
    """
    X_train_batch, X_val_batch, y_train_batch, y_val_batch = train_test_split(
        X, y, test_size=val_size, random_state=random_state, stratify=y
    )
    log_regression = LogisticRegression(
        max_iter=1000,
        solver="saga", # used for multiclass,
        n_jobs=-1,
        penalty='l2',
        class_weight = 'balanced'
    )

    n_samples, n_features = X_train_batch.shape
    selected = []
    remaining = list(range(n_features))

    while len(selected) < n_features_to_select and remaining:

      scores_this_round = []
      for feature_index in remaining:
          candidate_feature = selected + [feature_index]

          score = _evaluate_feature_subset(
              model = log_regression,
              X_train = X_train_batch,
              y_train = y_train_batch,
              X_valid = X_val_batch,
              y_valid = y_val_batch,
              feature_indices = candidate_feature,
              scoring = scoring,
```

```
            score_average = score_average
        )
        scores_this_round.append((feature_index, score))

    # pick best batch
    scores_this_round.sort(key=lambda x: x[1], reverse=True)
    best_batch = []
    for feat, score in scores_this_round[:batch_size]:
      best_batch.append(feat)
    selected.extend(best_batch)

    # get remaining fearutes (one that is not selected)
    remaining = [i for i in remaining if i not in best_batch]

    # print at what time features selected
    print(f'Selected at {time.strftime("%Y-%m-%d %H:%M:%S")}: {len(selected)}␣
↪/ {n_features_to_select}')
  return selected[:n_features_to_select]
```

[ ]:
```
# Common method to evaluate model
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import random
from sklearn.metrics import accuracy_score, f1_score, classification_report
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

def evaluate_model(model, model_name, X_train, y_train, X_test, y_test,␣
↪k_value):
    """
    Evaluates a machine learning model using training and testing data, and␣
↪prints various evaluation metrics including accuracy, classification report,␣
↪macro F1-score, and weighted F1-score. The function also returns the␣
↪predicted labels for the test dataset.
    """

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    print(f"\nEvaluation Metrics {model_name} chi2 k = {k_value} FS")
    print(f"{model_name} Accuracy:", accuracy_score(y_test, y_pred))
    print("Classification report:\n", classification_report(y_test, y_pred,␣
↪zero_division=0))
    print("Macro F1-score:", f1_score(y_test, y_pred, average="macro",␣
↪zero_division=0))
    print("Weighted F1-score:", f1_score(y_test, y_pred, average="weighted",␣
↪zero_division=0))
```

```
    return y_pred

def plot_confusion_matrices(model_name, y_test, y_pred, k_value):
  """
  Plots confusion matrices for Models
  """
  cmaps = [
    "Blues", "Greens", "Purples", "Oranges", "Reds",
    "viridis", "plasma", "inferno", "magma", "cividis",
    "YlGnBu", "YlOrRd", "PuBuGn", "GnBu", "BuPu",
  ]

  cmap_choice = random.choice(cmaps)

  cm = confusion_matrix(y_test, y_pred)
  disp = ConfusionMatrixDisplay(confusion_matrix=cm)
  disp.plot(cmap=cmap_choice)
  plt.title(f"Confusion Matrix - {model_name} chi2 {k_value} - FS Selection")
  plt.show()
```

- Executes a custom **forward feature selection** process to further reduce dimensionality.
- Trains and evaluates multiple classifiers (**Logistic Regression**, **Multinomial Naive Bayes**, **Decision Tree**, **Random Forest**) on the chosen features using the holdout test set.
- Compares model performance using F1 score and accurcy score
- Uses starfified K-Fold CV to evaluate and compare models

[50]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.metrics import accuracy_score, f1_score, classification_report
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay


# Run forward selection for features of different sizes selected after ch2
selected_idx = {}
final_feature_names = {}
final_feature_size = 1000

skfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

for key in X_test_chi2_selection.keys():
  print(f"Selected chi2 features: {X_train_chi2_selection[key].shape[1]} FS␣
 ↪features: {final_feature_size}")
  selected_idx[key] = batched_forward_selection(
```

```
    X_train_chi2_selection[key],
    y_train,
    n_features_to_select=final_feature_size,  # change if needed
    batch_size=50,
    val_size=0.1,
    random_state=42,
)
  # map to features

  final_feature_names[key] = selected_features[key][selected_idx[key]]

  X_train_after_fs = X_train_chi2_selection[key][:, selected_idx[key]]
  X_test_after_fs  = X_test_chi2_selection[key][:, selected_idx[key]]

  # Run logistic regression
  logistic_regression= LogisticRegression(max_iter=2000,
                        n_jobs=-1,
                        solver="saga",
                        penalty='l2', # improves reall for manority classes
                        class_weight = 'balanced')

  # Cross validation Score on Training data for Model comparison
  lr_scores = cross_val_score(
      logistic_regression,
      X_train_after_fs,
      y_train,
      cv=skfold,
      scoring='f1_macro',
      n_jobs=-1
  )

  print(f"CV Logistic Regression, (k{key}) F1 macro per fold:{lr_scores}")
  print(f"CV Logistic Regression, (k{key}) mean={lr_scores.mean()},␣
↪std={lr_scores.std()}")

  y_pred = evaluate_model(logistic_regression, "Logistic Regression",␣
↪X_train_after_fs, y_train, X_test_after_fs, y_test, key)
  plot_confusion_matrices("Logistic Regression", y_test, y_pred, key)

  # Run Naive Bayes model
  nb = MultinomialNB()
  nb.fit(X_train_after_fs, y_train)

  # Cross validation Score on Training data for Model comparison
  nb_scores = cross_val_score(
      nb,
      X_train_after_fs,
```

```
    y_train,
    cv=skfold,
    scoring='f1_macro',
    n_jobs=-1
)

print(f"CV Naive Bayes, (k{key}) F1 macro per fold:{nb_scores}")
print(f"CV Naive Bayes, (k{key}) mean={nb_scores.mean()}, std={nb_scores.
↪std()}")

y_pred = evaluate_model(nb, "Multinomial Naive Bayes", X_train_after_fs,␣
↪y_train, X_test_after_fs, y_test, key)
plot_confusion_matrices("Multinomial Naive Bayes", y_test, y_pred, key)

# Run decision Tree
dt = DecisionTreeClassifier(
    criterion="gini",   # or "entropy"
    max_depth=20,       # keep it from overfitting on text
    min_samples_leaf=5, # smooth leaves a bit
    random_state=42
)
# Cross validation Score on Training data for Model comparison
dt_scores = cross_val_score(
    dt,
    X_train_after_fs,
    y_train,
    cv=skfold,
    scoring='f1_macro',
    n_jobs=-1
)

print(f"CV Decision Tree, (k{key}) F1 macro per fold:{dt_scores}")
print(f"CV Decision Tree, (k{key}) mean={dt_scores.mean()}, std={dt_scores.
↪std()}")

y_pred = evaluate_model(dt, "Decision Tree", X_train_after_fs, y_train,␣
↪X_test_after_fs, y_test, key)
plot_confusion_matrices("Decision Tree", y_test, y_pred, key)

# Run Random forest classifier


rf = RandomForestClassifier(
    n_estimators=200,
    class_weight = 'balanced',
    max_depth=None,
    n_jobs=-1,
```

```
    random_state=42
)

# Cross validation Score on Training data for Model comparison
rf_scores = cross_val_score(
    rf,
    X_train_after_fs,
    y_train,
    cv=skfold,
    scoring='f1_macro',
    n_jobs=-1
)

print(f"CV Decision Tree, (k{key}) F1 macro per fold:{rf_scores}")
print(f"CV Decision Tree, (k{key}) mean={rf_scores.mean()}, std={rf_scores.
↪std()}")

y_pred = evaluate_model(rf, "Random Forest", X_train_after_fs, y_train,␣
↪X_test_after_fs, y_test, key)
plot_confusion_matrices("Random Forest", y_test, y_pred, key)
```

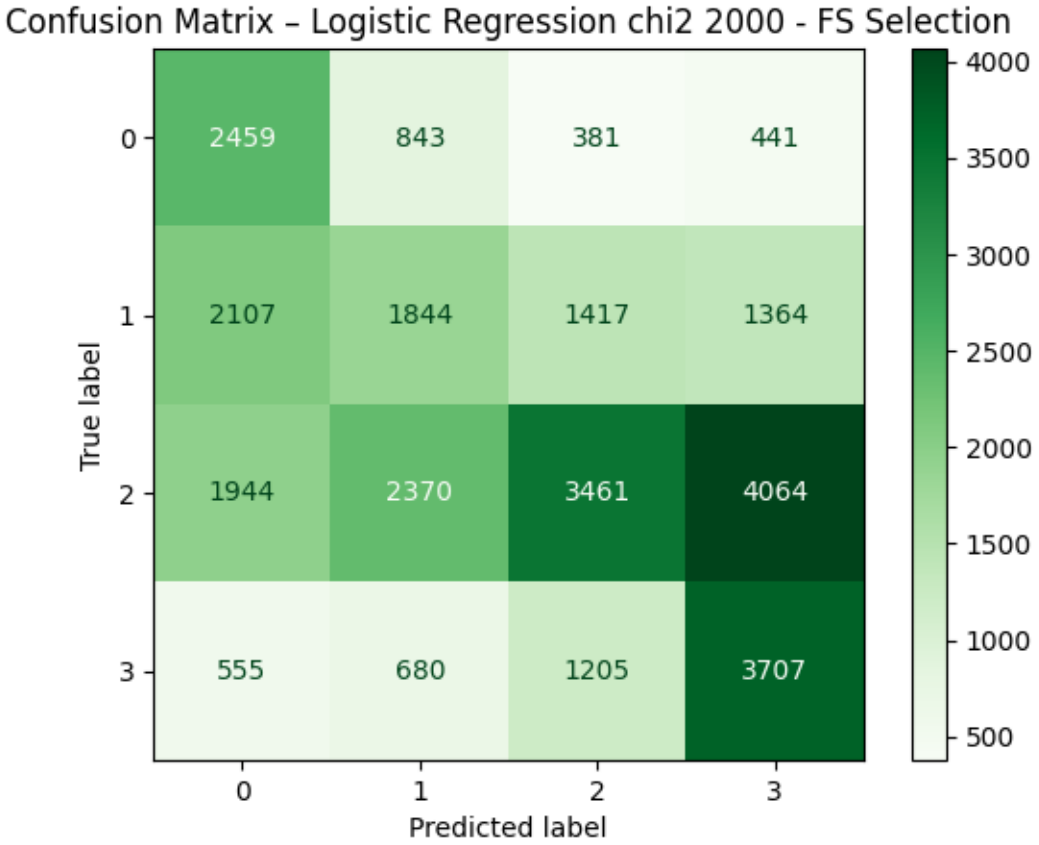
```
Selected chi2 features: 2000 FS features: 1000
Selected at 2025-11-30 08:12:45: 50 / 1000
Selected at 2025-11-30 08:32:21: 100 / 1000
Selected at 2025-11-30 09:09:06: 150 / 1000
Selected at 2025-11-30 09:47:51: 200 / 1000
Selected at 2025-11-30 10:29:31: 250 / 1000
Selected at 2025-11-30 11:12:32: 300 / 1000
Selected at 2025-11-30 11:56:35: 350 / 1000
Selected at 2025-11-30 12:41:28: 400 / 1000
Selected at 2025-11-30 13:26:01: 450 / 1000
Selected at 2025-11-30 14:10:06: 500 / 1000
Selected at 2025-11-30 14:56:36: 550 / 1000
Selected at 2025-11-30 15:42:20: 600 / 1000
Selected at 2025-11-30 16:27:13: 650 / 1000
Selected at 2025-11-30 17:11:23: 700 / 1000
Selected at 2025-11-30 17:55:09: 750 / 1000
Selected at 2025-11-30 18:38:01: 800 / 1000
Selected at 2025-11-30 19:19:22: 850 / 1000
Selected at 2025-11-30 19:58:52: 900 / 1000
Selected at 2025-11-30 20:36:27: 950 / 1000
Selected at 2025-11-30 21:13:39: 1000 / 1000
Final feature names  5000:['get' 'people' 'customer' 'need' 'work' 'clean'
'employee' 'good' 'drive'
 'finally' 'front' 'guy' 'business' 'store' 'wrong' 'customer service'
 'today' 'pick' 'salsa' 'literally' 'behind' 'burrito' 'closed' 'car'
 'lady' 'taking' 'wrap' 'working' 'get food' 'machine' 'cashier' 'running'
```

'chipotle' 'standing' 'credit' 'cheesecake' 'correct' 'sub' 'mall'
'bagel' 'speak' 'airport' 'young' 'one time' 'pull' 'little place'
'register' 'mistake' 'got food' 'soda' 'best' 'delicious' 'amazing'
'coffee' 'sandwich' 'owner' 'try' 'definitely' 'flavor' 'friendly'
'fresh' 'food' 'service' 'everything' 'place' 'like' 'time' 'menu'
'drink' 'ordered' 'server' 'bar' 'friend' 'beer' 'came' 'always' 'dinner'
'one' 'order' 'happy hour' 'hour' 'happy' 'pizza' 'every' 'market'
'table' 'price' 'pretty' 'incredible' 'horrible' 'oyster' 'decent'
'atmosphere' 'shrimp' 'didnt' 'enjoyed' 'special' 'ice' 'french' 'give'
'spot' 'great' 'recommend' 'space' 'fantastic' 'nashville' 'dish' 'local'
'indian' 'small' 'perfect' 'wait' 'pulled pork' 'roll' 'chip' 'loved'
'indian food' 'date' 'beignet' 'band' 'thai' 'unique' 'thank' 'spicy'
'gem' 'wow' 'potato' 'mexican restaurant' 'pulled' 'past' 'phenomenal'
'one best' 'top' 'taco' 'guacamole' 'interesting' 'quaint' 'hidden'
'vegan' 'greek' 'stout' 'ive ever' 'five guy' 'list' 'cocktail' 'crepe'
'truly' 'pairing' 'burger' 'chocolate' 'waitress' 'great food' 'manager'
'pat' 'side' 'genos' 'dont' 'definitely back' 'particular' 'dollar' 'tea'
'applebees' 'chain' 'casino' 'movie' 'panera' 'check' 'waited' 'stay'
'margarita' 'paid' 'tv' 'city' 'game' 'mess' 'potato salad' 'great place'
'night' 'waste' 'recommended' 'pool' 'brisket' 'go wrong' 'robin'
'consistent' 'fry' 'location' 'empty' 'outstanding' 'next time'
'took minute' 'casual' 'shack' 'going' 'hit miss' 'paper' 'suck' 'worth'
'staff super' 'card' 'wine' 'wonderful' 'tampa' 'pork' 'dessert'
'seating' 'appetizer' 'grit' 'cant' 'brunch' 'truck' 'duck' 'perfectly'
'knowledgeable' 'creative' 'style' 'life' 'anniversary' 'sweet'
'much better' 'st' 'hipster' 'glad' 'also' 'looked' 'short rib' 'evening'
'food truck' 'sangria' 'wine selection' 'absolutely' 'chili' 'lamb'
'arrived' 'grouper' 'tapa' 'tour' 'tasting menu' 'cafe' 'heat'
'reservation' 'foodie' 'pickled' 'forward' 'cute' 'chicken waffle' 'deli'
'fabulous' 'favorite restaurant' 'guide' 'buffet' 'music' 'live music'
'chinese' 'view' 'sushi' 'crab' 'fried chicken' 'nothing' 'water' 'bland'
'attentive' 'buck' 'noodle' 'waiter' 'crawfish' 'pudding' 'said'
'hibachi' 'average' 'ask' 'seemed' 'someone' 'general' 'patio'
'food good' 'grill' 'every dish' 'eggplant' 'usually' 'somewhere'
'ambiance' 'commander' 'star' 'nacho' 'seafood' 'shrimp cocktail'
'wait staff' 'bean' 'awful' 'subway' 'po boy' 'wont' 'seem' 'inedible'
'dj' 'best fried' 'isnt' 'call' 'savory' 'nola' 'breakfast' 'bloody'
'pancake' 'bloody mary' 'little' 'mary' 'fast food' 'bartender' 'egg'
'chinese buffet' 'bag' 'dennys' 'love' 'favorite' 'super' 'philly'
'overpriced' 'morning' 'portion' 'perfection' 'finger' 'hash'
'way overpriced' 'hurricane' 'hard rock' 'mein' 'deep dish' 'tomato'
'sprout' 'belly' 'sour' 'delivery' 'classy' 'ever eaten' 'benedict'
'egg benedict' 'brussels' 'new orleans' 'management' 'receipt' 'orleans'
'sour cream' 'food tasted' 'hot chicken' 'care' 'mussel' 'lost' 'sum'
'monk' 'room' 'ever' 'mcdonalds' 'meat' 'bad' 'disgusting' 'floor'
'fajitas' 'disappointing' 'took' 'dirty' 'beet' 'gross' 'wing'
'nothing special' 'worse' 'cleaned' 'pub' 'last time' 'hummus' 'diner'
'another' 'highly recommended' 'gelato' 'took another' 'new' 'option'

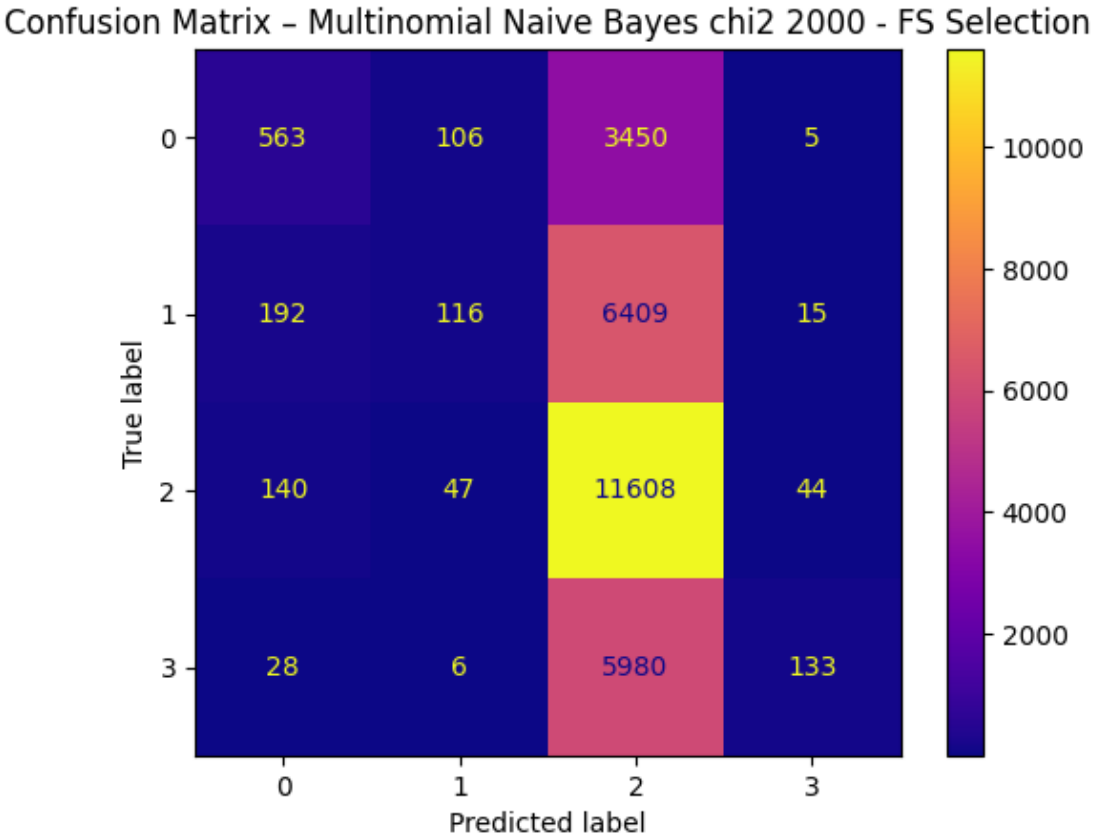
'broth' 'bread' 'fondue' 'wasnt great' 'barely' 'cinnamon' 'even'
'food came' 'burger joint' 'help' 'steak shake' 'pm' 'burnt' 'college'
'ambience' 'coffee shop' 'girl' 'ice cream' 'horrendous' 'came back'
'coupon' 'trip' 'terminal' 'mac' 'fish' 'gone' 'fish chip' 'watching'
'hype' 'mac cheese' 'flavorful' 'takeout' 'counter' 'soft' 'pho'
'mediocre' 'quesadilla' 'charged' 'jims' 'outback' 'pita' 'authentic'
'lettuce' 'hair' 'service slow' 'crust' 'butter' 'front desk' 'know'
'pay' 'tasting' 'either' 'groupon' 'ihop' 'small place' 'poboy'
'cauliflower' 'prime' 'act' 'tasteless' 'ago' 'dont understand'
'bathroom' 'pad' 'poboys' 'food excellent' 'salmon' 'joes' 'subpar'
'sent back' 'creme brulee' 'fresh ingredient' 'crispy' 'however'
'baked good' 'hattie' 'korean' 'never' 'dough' 'dip' 'broccoli' 'hoagie'
'strip mall' 'avocado' 'enchilada' 'pool table' 'sell' 'phone'
'family owned' 'hooter' 'poorly' 'roast beef' 'great beer' 'cart'
'selection great' 'wasnt' 'butcher' 'cinnamon roll' 'pastrami' 'tequila'
'spinach artichoke' 'wonton soup' 'food wonderful' 'best restaurant'
'went' 'divine' 'quiche' 'wont going' 'everything great' 'whipped'
'owned' 'factory' 'never go' 'chicken strip' 'great flavor' 'found place'
'muffuletta' 'outside' 'al pastor' 'royal' 'typical' 'cant go'
'place used' 'pappys' 'zahav' 'minute' 'excellent' 'last' 'took min'
'stop' 'espresso' 'loud' 'neighborhood' 'wait minute' 'guess'
'shrimp grit' 'perfectly cooked' 'acknowledged' 'get order'
'service excellent' 'everyone' 'pastry' 'baklava' 'bbq shrimp' 'latte'
'love love' 'treat' 'cuban' 'southern' 'soap' 'place great' 'mex'
'seitan' 'burnt end' 'rib' 'walked' 'cappuccino' 'kabob' 'kid meal'
'almond' 'dog' 'authentic food' 'el vez' 'vez' 'passionate' 'agave'
'excellent food' 'minute food' 'one favorite' 'min later' 'back soon'
'deviled egg' 'matcha' 'hotel' 'impeccable' 'cracker barrel' 'rude'
'cake' 'station' 'amazing food' 'anymore' 'probably wont' 'lovely'
'best bbq' 'silver' 'ever go' 'wawa' 'improvement' 'franchise' 'occupied'
'fave' 'wouldnt' 'thru' 'decor' 'corporate' 'fine' 'seated' 'thats'
'delightful' 'nice atmosphere' 'new favorite' 'party' 'charge'
'food cold' 'microwave' 'menu change' 'rip' 'great recommendation'
'friendly helpful' 'salad bar' 'toast' 'stl' 'sad' 'always clean' 'gave'
'desk' 'cover' 'hole wall' 'club' 'shut' 'adult' 'shouldve' 'han' 'tasty'
'line' 'gluten free' 'gumbo' 'slow service' 'gluten' 'waffle'
'love place' 'homemade' 'naan' 'boy' 'world' 'draft' 'cupcake' 'addition'
'best breakfast' 'donut' 'starbucks' 'rather' 'falafel' 'amazing service'
'garlic' 'everything fresh' 'santa barbara' 'pier' 'craft' 'share'
'cookie' 'wanting' 'lettuce wrap' 'ramen' 'smell like' 'welcoming'
'plate' 'bother' 'ingredient' 'beer list' 'great review' 'decadent'
'place small' 'half' 'inviting' 'hookah' 'delight' 'spa' 'byob' 'art'
'rabbit' 'louis' 'blah' 'terrible' 'cold' 'worst' 'kid' 'mongolian'
'mexican food' 'stick' 'still' 'brought' 'bad review' 'cheap' 'problem'
'child' 'delivered' 'domino' 'grilled' 'frozen' 'crappy' 'greasy' 'slow'
'food mediocre' 'issue' 'person' 'express' 'received' 'later' 'popeyes'
'mediocre best' 'wait go' 'poor' 'convenient' 'circus' 'thrown' 'king'
'everyday' 'bar food' 'olive garden' 'terrible food' 'expect'

'usually pretty' 'drunk' 'moes' 'good service' 'jambalaya' 'refill'
'normal' 'late' 'crap' 'food delicious' 'grease' 'tried' 'cuisine'
'everything menu' 'par' 'hostess' 'quarter' 'beautifully' 'amazing staff'
'date night' 'bun' 'dance' 'carrot' 'qdoba' 'even better' 'french toast'
'flaky' 'bar area' 'french quarter' 'bad food' 'order counter'
'great vibe' 'complaining' 'deck' 'lighting' 'informed' 'tap'
'chargrilled oyster' 'shouldnt' 'boise' 'never return' 'wait come'
'dont let' 'minute get' 'back try' 'dont care' 'shake shack' 'tower'
'dont know' 'great coffee' 'bowl' 'service impeccable' 'rooftop'
'favorite place' 'delicious great' 'roast pork' 'sabrinas' 'thai food'
'food took' 'superb' 'fried oyster' 'basic' 'online' 'calamari'
'vegan option' 'leaving' 'cheek' 'worker' 'puff' 'called' 'email'
'ethiopian' 'modern' 'dan' 'wont regret' 'order online' 'food ok'
'food actually' 'trendy' 'ruth' 'unless' 'better option' 'mimosa'
'napkin' 'jackson' 'drink order' 'mojitos' 'avoid' 'cant speak' 'banana'
'tartare' 'fresh baked' 'delicate' 'trash' 'unique flavor' 'late night'
'chinese place' 'tritip' 'taking order' 'chickfila' 'kebab' 'bahn'
'adequate' 'ridiculous' 'green sauce' 'booth' 'monkey' 'fault'
'minute later' 'frites' 'poke' 'coke' 'dont go' 'bakery' 'three star'
'cute place' 'cold food' 'chain restaurant' 'pitcher' 'ceviche'
'pat genos' 'beer cold' 'two star' 'cook' 'service okay' 'pesto' 'dated'
'yuck' 'sunset' 'understaffed' 'kfc' 'best ive' 'wait worth' 'mad'
'hour half' 'great service' 'people dont' 'stuck' 'pina' 'prime rib'
'not wait' 'guinness' 'left' 'placed order' 'brussels sprout'
'best burger' 'full flavor' 'split' 'elses' 'drivethru' 'whiz' 'golf'
'place empty' 'wasnt even' 'carpet' 'came cold' 'outdoor' 'blood orange'
'joke' 'kale' 'write' 'soup dumpling' 'reuben' 'wish' 'beautiful' 'vibe'
'traditional' 'shake' 'juicy' 'empanadas' 'sandwich shop' 'blame'
'service terrible' 'fig' 'berry' 'drink special' 'pf' 'chang'
'food amazing' 'pf chang' 'melt' 'acai' 'nearly' 'aioli' 'toilet' 'bill'
'restroom' 'menu small' 'banh' 'red robin' 'visiting' 'flavorless'
'banh mi' 'nope' 'blueberry' 'horribly' 'played' 'expect much' 'truffle'
'twenty' 'complain' 'dont miss' 'tip' 'dive' 'service hit' 'house made'
'mass ave' 'tri' 'prosciutto' 'lacking' 'asking' 'least' 'seems'
'service great' 'used' 'job' 'sick' 'cost' 'tank' 'iced' 'bell' 'filling'
'heaven' 'ordered drink' 'took order' 'milkshake' 'ruby' 'cheeseburger'
'muffaletta' 'pork chop' 'cracker' 'dozen' 'time ordered' 'standard'
'calling' 'seen' 'place yelp' 'salsa bar' 'apparently' 'cheese steak'
'comment' 'breakfast sandwich' 'goat cheese' 'ranch' 'fluffy' 'rabe'
'check place' 'annoying' 'department' 'suite' 'elsewhere' 'wit'
'place watch' 'asada' 'zero' 'eh' 'extra' 'potato fry' 'cheesesteak'
'response' 'risotto' 'chef' 'ignored' 'use' 'chip salsa' 'gratuity'
'stale' 'tuesday' 'passion' 'tab' 'food poisoning' 'previous' 'boba'
'worst experience' 'sausage' 'lot tv' 'gin' 'devil' 'money' 'wiz'
'cant wait' 'wifi' 'caramelized' 'hamburger' 'gnocchi' 'buffalo wing'
'gorgeous' 'green' 'definitely worth' 'creme' 'decent food' 'isnt great'
'panda' 'bjs' 'watched' 'fried green' 'rubber' 'slot' 'speak manager'
'would definitely' 'vanilla' 'doughnut' 'poisoning' 'french fry'

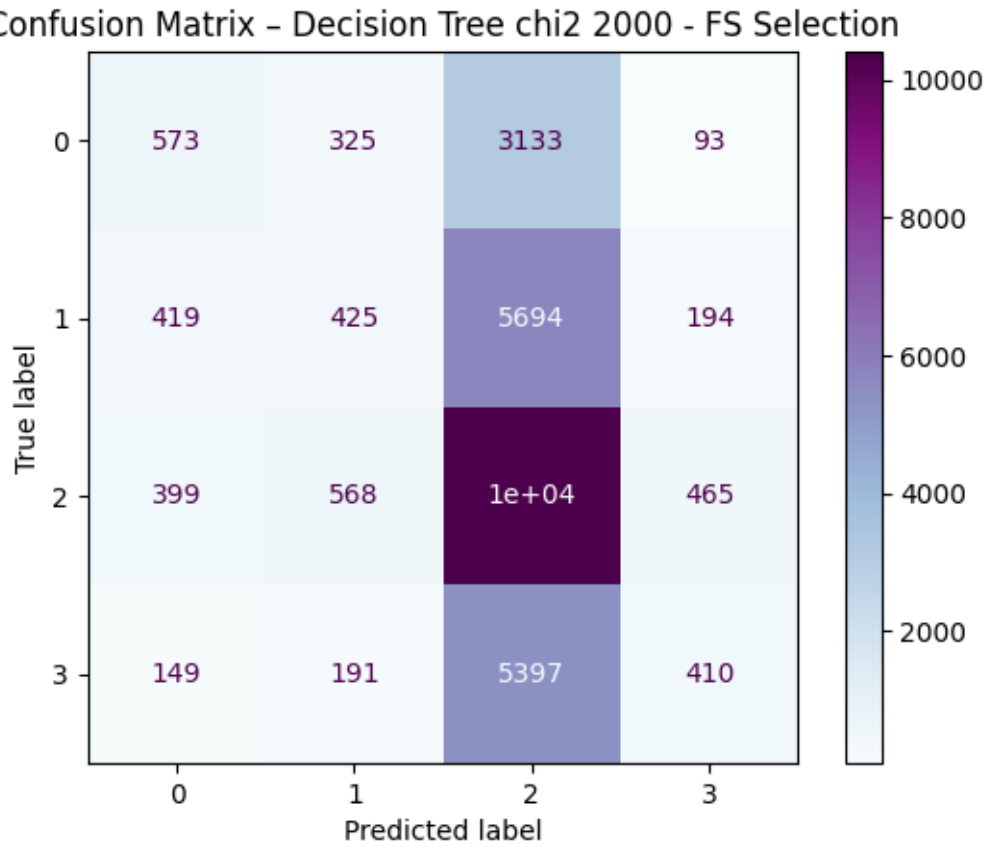
```
 'organic' 'waitress took' 'coconut' 'screw' 'hire' 'attempted' 'fixed']
CV Logistic Regression, (k2000) F1 macro per fold:[0.40293498 0.40005808
0.39980341 0.40732633 0.40098935]
CV Logistic Regression, (k2000) mean=0.4022224290774837, std=0.00277899390605656

Evaluation Metrics Logistic Regression chi2 k = 2000 FS=300
Logistic Regression Accuracy: 0.3977186048124263
Classification report:
              precision    recall  f1-score   support

           1       0.35      0.60      0.44      4124
           2       0.32      0.27      0.30      6732
           3       0.54      0.29      0.38     11839
           4       0.39      0.60      0.47      6147

    accuracy                           0.40     28842
   macro avg       0.40      0.44      0.40     28842
weighted avg       0.43      0.40      0.39     28842

Macro F1-score: 0.39626005735830394
Weighted F1-score: 0.3876200201686869
```



```
CV Naive Bayes, (k2000) F1 macro per fold:[0.23160292 0.22910145 0.22595457
0.22749328 0.22633311]
CV Naive Bayes, (k2000) mean=0.2280970657331723, std=0.002067183208935423

Evaluation Metrics Multinomial Naive Bayes chi2 k = 2000 FS=300
Multinomial Naive Bayes Accuracy: 0.430622009569378
Classification report:
              precision    recall  f1-score   support

           1       0.61      0.14      0.22      4124
           2       0.42      0.02      0.03      6732
           3       0.42      0.98      0.59     11839
           4       0.68      0.02      0.04      6147

    accuracy                           0.43     28842
   macro avg       0.53      0.29      0.22     28842
weighted avg       0.50      0.43      0.29     28842

Macro F1-score: 0.22227259808027547
Weighted F1-score: 0.2911361650525035
```



```
CV Decision Tree, (k2000) F1 macro per fold:[0.2523584  0.25428748 0.23384013
0.24662694 0.24913025]
CV Decision Tree, (k2000) mean=0.24724863919048926, std=0.007202363040143126

Evaluation Metrics Decision Tree chi2 k = 2000 FS=300
Decision Tree Accuracy: 0.40964565564107897
Classification report:
               precision    recall  f1-score   support

           1       0.37      0.14      0.20      4124
           2       0.28      0.06      0.10      6732
           3       0.42      0.88      0.57     11839
           4       0.35      0.07      0.11      6147

    accuracy                           0.41     28842
   macro avg       0.36      0.29      0.25     28842
weighted avg       0.37      0.41      0.31     28842

Macro F1-score: 0.24709485944542356
Weighted F1-score: 0.3111818131373233
```



```
CV Decision Tree, (k2000) F1 macro per fold:[0.30539833 0.31590311 0.3187069
0.30884956 0.30334161]
CV Decision Tree, (k2000) mean=0.3104399010935925, std=0.005941757349428263

Evaluation Metrics Random Forest chi2 k = 2000 FS=300
Random Forest Accuracy: 0.4380764163372859
Classification report:
              precision    recall  f1-score   support

           1       0.50      0.26      0.34      4124
           2       0.33      0.07      0.11      6732
           3       0.44      0.87      0.58     11839
           4       0.46      0.12      0.19      6147

    accuracy                           0.44     28842
   macro avg       0.43      0.33      0.31     28842
weighted avg       0.43      0.44      0.36     28842

Macro F1-score: 0.30809680218027646
Weighted F1-score: 0.35593314767982526
```